

Posibles preguntas y cómo responderlas

Análisis de los puntos débiles del proyecto · Para los tres

Preguntas ordenadas de más probable a menos. Las marcadas como **DIFÍCIL** son las que de verdad pueden complicarles la defensa.

Sobre los resultados

DIFÍCIL *¿Por qué el agente pierde tanto? Solo gana el 31% de las 800 corridas.*

Ese 31% es el promedio de las 800 y esconde la historia. En duelo 1v1 gana entre el 62% y el 81% contra los tres enemigos FSM. Lo que arrastra el promedio son los escenarios de desventaja numérica: uno contra tres y uno contra seis, donde cae a cero. Es un resultado, no un fallo: el agente aprendió a pelear en duelo y no aprendió a manejar el ser rodeado.

Responde: Raúl

DIFÍCIL *Contra el Tipo A gana el 62% pero mata 0,00. ¿Cómo gana entonces?*

No gana él: el enemigo se mata solo en las púas mientras lo persigue. El agente hace 4,0 de daño por segundo, o sea que pega, pero no remata. El propio informe cualitativo marca esa fila con un aviso para que no se lea como desempeño del agente. La fila comparable es la del Tipo B: 68,8% de victorias con 0,69 kills, ahí sí lo mata.

Responde: Raúl

¿Por qué la tasa de éxito del Tipo C es 0,12 si gana el 81% de las partidas?

Por lo mismo. El Tipo C huye, y huyendo pisa púas. El agente gana la partida sin haber matado. Por eso reportamos victorias y tasa de éxito por separado en vez de una sola cifra: una mide quién quedó vivo, la otra si el agente fue la causa.

Responde: Raúl

¿25 generaciones y población 20 no es muy poco?

Sí, es poco para maximizar el agente. Fue una decisión de reparto: el enunciado pide comparar ocho variables, así que preferimos 16 configuraciones entrenadas a 25 generaciones antes que una sola configuración entrenada a 400. La curva de convergencia de cada configuración está en los archivos `convergencia_*.json`; se ve que varias aún subían al cortar.

Responde: Raúl

Sobre el método

DIFICIL ¿Esto es aprendizaje reforzado o es otra cosa?

Es neuroevolución. En la página 4 de su enunciado usted lo plantea así: aprendizaje por refuerzo implícito en los algoritmos genéticos. La recompensa es la función de fitness y la señal llega por selección, no por retropropagación. No hay Q-learning ni replay buffer, y no los hay a propósito, porque el enunciado pide optimizar los pesos con algoritmos genéticos.

Responde: Noryel

DIFICIL Ustedes ajustaron la función de fitness hasta que dio buenos números. ¿No es hacer trampa?

Ajustamos la función porque la primera estaba mal, y sabemos exactamente en qué. Premiaba sobrevivir a 0,015 por tick; con episodios de 1.800 ticks, un agente que no hacía nada puntuaba 27,00. Como pelear arriesga daño y muerte, toda la población convergió a agentes-estatua: en el log salía «mejor 27,00, medio 27,00» durante etapas enteras, o sea cero presión selectiva. No ajustamos para subir un número: corregimos un óptimo local que nos estaba dando un agente inútil. Los pesos y el porqué de cada uno están documentados en el código.

Responde: Noryel

¿Por qué OFAT y no un barrido factorial completo?

Por coste. El factorial completo de esas ocho variables son 2.592 entrenamientos; OFAT son 16 y responde la pregunta que pide el enunciado, que es el efecto de cada variable. Está declarado como limitación: medimos efectos marginales, no interacciones. Las dos tablas de ejemplo del propio enunciado tienen filas de valores sueltos, no de combinaciones.

Responde: Raúl

DIFICIL ¿Cómo sé que acelerar el entrenamiento no cambió los resultados?

Porque la simulación no usa el delta del motor: avanza con un paso fijo y contadores de ticks enteros, y los actores se mueven con `move_and_collide`, no con `move_and_slide`, que leería ese delta. Además medimos que `Engine.time_scale` a 40 daba una aceleración real de solo 1,2, así que aceleramos subiendo la frecuencia de física, que sí multiplica los pasos y es seguro justamente por el paso fijo.

Responde: Cristian

¿Cómo demuestran que es reproducible?

Corrimos dos veces la misma configuración con la misma semilla y salieron cifras idénticas. Y no era gratis: hasta dieciséis combates corren en paralelo, cada uno con su propio espacio de física y su propio generador de aleatoriedad. Con un generador global compartido, las arenas consumen el mismo flujo intercalado y la misma semilla deja de dar el mismo resultado.

Responde: Raúl

Sobre la IA del agente

DIFICIL El agente recibe distancia y ángulo aunque haya una pared. ¿No está viendo a través de los muros?

Sí, y está declarado como limitación. La oclusión va en un sensor aparte, el de línea de visión, así que el agente sabe si tiene tiro limpio. Es la lista de sensores que fija el enunciado, pero implica que no tiene que resolver búsqueda con información parcial: sabe dónde está el objetivo aunque no lo vea. Los enemigos FSM sí tienen esa restricción, porque usan cono de visión con memoria corta.

Responde: Noryel

El agente casi no usa «Huir» y el Tipo C sí. ¿No aprendió esa acción?

Correcto, y el informe lo dice: usó la huida en el 0,4% de sus decisiones contra el Tipo A y el 0,2% contra el Tipo C. Su política es plantarse y pelear. Con la función de fitness actual huir no paga, porque el término que premia estar en rango de combate empuja en sentido contrario. Es un resultado medido, no un descuido.

Responde: Noryel

¿Cómo saben que congelar sensores sirvió de algo?

Honestamente, no lo aislamos como variable del barrido: lo aplicamos porque usted lo recomienda en IV.2 y porque el caso se da de verdad, el sensor de escudo vale cero en todas las etapas contra Tipo A y Tipo C, que no pueden defender. Medir su efecto exigiría dos corridas completas con y sin congelación, y sería la primera ampliación que haríamos.

Responde: Noryel

¿Por qué la red decide cada 6 ticks y no cada tick?

Por coherencia y por coste. A 60 Hz el agente cambiaría de acción 60 veces por segundo y produciría el temblor típico de las políticas sin memoria: avanza-huye-avanza sin desplazarse. A 10 Hz las acciones duran lo suficiente para que se note su efecto, que es justo lo que el fitness necesita medir. Y el forward pass es la parte cara del entrenamiento.

Responde: Noryel

Sobre el juego

Muéstrenme la máquina de estados funcionando.

F1 en cualquier nivel. Sobre cada enemigo aparece su estado actual, su vida, sus radios de transición y la línea de visión al objetivo, verde o roja según si lo ve.

Responde: Cristian

¿Revisaron el ejemplo que les compartí?

Sí. Es un proyecto de GameMaker. Su patrón central es que el jugador y el enemigo comparten el mismo bloque de movimiento y solo cambia `get_input`: teclado para uno, variables escritas por la FSM para el otro. Nosotros tenemos esa misma idea un nivel más arriba: `move_in_direction` es la tubería compartida y el `think` de cada clase es el cerebro intercambiable. De su ejemplo adoptamos literalmente el overlay de depuración.

Responde: Cristian

DIFICIL ¿Dónde están las partidas contra un jugador humano de verdad?

Los escenarios 4 y 10 del barrido usan un bot sustituto, porque una persona no cabe en 800 partidas automatizadas, y lo declaramos como limitación. Las partidas contra humano real las jugamos aparte y las traemos anotadas.

Responde: Raúl

¿Qué pasa si les cambio el nivel?

La evaluación ya corre en un nivel que el agente nunca vio: entrenamos en tres niveles y medimos en un cuarto, de validación, precisamente para no medir memoria del mapa. Añadir otro nivel es escribir un mapa ASCII; el resto del sistema no cambia.

Responde: Cristian

La pregunta abierta

¿Qué harían distinto si tuvieran más tiempo?

Tres cosas, por orden. Primero, más generaciones: varias curvas aún subían al cortar. Segundo, aislar el efecto de congelar sensores como una variable más del barrido. Tercero, atacar la debilidad que el propio benchmark destapó: el agente se hunde en desventaja numérica, y no lo entrenamos contra grupos, solo contra oponentes de uno en uno. Entrenarlo en escenarios de varios contra uno es la mejora con más recorrido.

Responde: cualquiera